LESSON 03 · MISSION GUIDE

Dave's Spaghetti

Write a rule once, give it a name, use it everywhere: functions.

From **Mo, IT**
To **Jen**
Subject **I opened MACRO-9000 (don't)**

Jen. I opened MACRO-9000 to find out why the bonus total changes depending on which button you press. The bonus rule is in there **forty-one times**. Copy-pasted. Thirty-eight of the copies agree with each other.

Dave's comment at the top says `'TODO: make this a function (what is a function)`. I've been staring at that line for six years.

Today you fix the thing Dave couldn't: write the rule once, name it, and call it from wherever it's needed. When Gerald changes the napkin, you change one place. Biscuit says hi. — Mo

What you'll be able to do after today

- [ ] Define a function with `def`, give it parameters, and hand a result back with `return`
- [ ] Call your own functions from other functions, so a rule lives in exactly one place
- [ ] Give parameters sensible **defaults**, and call functions by position or by keyword
- [ ] Explain the difference between `return` and `print`, and why a function "returned nothing"
- [ ] Know what a variable inside a function can and can't see (scope)

Why this matters at work

- **One rule, one place.** When the napkin changes, you change one function and every report follows. Copies drift; MACRO-9000 is the proof.
- **Named steps read like the policy.** `bonus(salary, months, rating)` says what it does. Forty lines of arithmetic don't.
- **Small functions can be checked one at a time** in the REPL. That's how you'll test the pay gap report later: piece by piece.

Four big ideas

1. `def` defines, calling runs	Writing `def money(amount):` teaches Python a recipe. Nothing happens until you call `money(50)`. VBA: `Function … End Function`.
2. `return` hands back, `print` shows	`return x` is VBA's `MoneyText = x`: the caller gets the value and can use it. `print` only puts it on screen. A function with no `return` hands back `None`.
3. Parameters are new local names	Inside the function, `amount` is whatever was passed in. Variables made inside stay inside. The outside world isn't changed.
4. Defaults and keywords	`def percent(x, decimals=1)`: call it as `percent(0.1)`, `percent(0.1, 2)` or `percent(0.1, decimals=2)`. Keywords make calls readable.

Three habits that will bite you

- Printing inside the function instead of returning, then wondering why `money(x) + "!"` fails: it's `None`.
- Forgetting the brackets when calling: `money` is the recipe, `money(50)` is the result.
- Defining the function *below* the line that calls it. Python reads top to bottom: `def` first, calls after.

How today works

15 MIN
Lecture
Lars turns last week's bonus loop into functions, live

60 MIN
Exercise sheet
Seven tickets. The calls are given; you write the functions

15 MIN
Dave's Leftovers #3
The functions that "returned nothing"

The inbox

#	FROM	TICKET	YOU'LL USE
1	Priya	Your first function	`def`, `return`, docstring
2	Siobhan	The napkin, as a function	early `return`, functions calling functions
3	Gerald	Formatting helpers	defaults, nested f-strings
4	Priya	One payslip, one function	keyword arguments, a function that prints
5	Priya	Holiday allowance, properly	small functions composed
6	Mo	What happens in a function stays there	scope
7 ⭐	Gerald	The pay gap toolkit	all of the above

You're done when...

- [ ] `my_work/` has `ex1` to `ex6` (and `ex7` if you're feeling brave), each printing the expected output
- [ ] Dave's helpers return what they should, with one commit per bug
- [ ] Everything is pushed to `lesson-03` and you've opened a pull request



Next lesson: The Staff List. So far every employee has been a line of text you split by hand. Next time Python gets proper containers: lists of people, and dictionaries that hold a whole employee record by name.